

SE/COM S 3190 - Construction of User Interfaces

**Final Project Technical Report
Spring 2026**

Outfitly

Team Members

Mokshitha Iragamreddy
mokshi28@iastate.edu

Pulkit Saxena
pulkit@iastate.edu

*Iowa State University
Department of Computer Science*

Contents

1 Introduction	3
1.1 Project Overview	3
1.2 Target Users and Use Cases	3
1.3 Core Features	3
1.4 Originality vs Inspiration	3
2 System Overview and Project Description	4
2.1 Major Functional Modules	4
2.2 UI and Navigation Flow	4
2.3 CRUD Entities	4
3 File and Folder Architecture	5
3.1 Folder Structure Overview	5
3.2 Backend Folder Structure	5
3.2.1 Backend Folder Tree	5
3.2.2 Backend Folder Description	5
3.3 Frontend Folder Structure	5
3.3.1 Frontend Folder Tree	5
3.3.2 Frontend Folder Description	6
4 Code Explanation and Logic Flow	6
4.1 Frontend–Backend Interaction	6
4.2 React Component Example	6
4.3 API Route Examples	7
4.4 Database Interaction	7
5 Screenshots and Web Views	7
6 Installation and Setup Instructions	8
6.1 Frontend Setup	8
6.2 Backend Setup	8
6.3 Database Setup	8
6.4 Environment Variables	8
7 Contribution Overview	9
7.1 Pulkit Contributions	9
7.2 Mokshitha Contributions	9
8 Challenges Faced	10
8.1 Pulkit Challenges	10
8.2 Mokshitha Challenges	10
9 Final Reflections	10
9.1 Pulkit Reflection	10
9.2 Mokshitha Reflection	10

1 Introduction

1.1 Project Overview

The PS-16 Outfit Planner and Wardrobe Management System is a full-stack web application designed to help users organize clothing items, create outfits, save favorite looks, and plan outfits for future dates. Many users have clothing items spread across different categories but do not have a simple way to manage them digitally. This can make it harder to decide what to wear, repeat favorite combinations, or plan outfits for school, events, interviews, trips, or daily routines.

The problem this application solves is the lack of a simple, organized, and user-friendly system for managing a personal closet and outfit planning workflow. Instead of remembering clothing combinations manually, users can store items, build outfits, view outfit details, mark outfits as favorites, and assign outfits to dates in a planner.

At a high level, this project is an interactive web application with a React frontend, an Express/Node.js backend, and a MongoDB database. The scope of the project includes user-facing pages, admin and authentication-related features, CRUD API endpoints, database models, and a connected frontend-backend workflow

1.2 Target Users and Use Cases

The primary users of this system are students and everyday users who want an easier way to organize their clothing and plan outfits. This application is especially useful for users who want to save time deciding what to wear and who want a visual or structured way to manage their wardrobe.

Typical use cases include:

- A user adds clothing items to their closet by category.
- A user creates an outfit by combining different clothing items.
- A user opens an outfit detail page to view, edit, or delete an outfit.
- A user marks an outfit as a favorite for quick access later.
- A user assigns an outfit to a calendar date for planning.

- An admin or project maintainer can manage data categories and verify stored information.

The secondary users are team members, graders, or administrators who may test the application, inspect the database, or review the CRUD behavior across the system.

1.3 Core Features

The system includes the following major features:

- **Closet and Item Management:**

Users can manage clothing items stored in the database. Items may include fields such as name, category, color, image, notes, or other descriptive information depending on the database schema.

- **Category Management:**

Clothing items can be organized into categories so that users can build outfits in a structured way. Examples may include tops, bottoms, shoes, accessories, and outerwear.

- **Outfit Management:**

Users can create, view, update, and delete outfits. This is one of the main CRUD features of the application. The outfit feature connects the Outfit Builder Page and Outfit Detail Page to backend API routes.

- **Favorites:**

Users can save outfits as favorites and remove them from favorites later. This makes it easier to quickly find preferred outfits.

- **Outfit Planner:**

Users can assign outfits to specific dates with optional notes or occasion details. This allows the application to work as both a closet management system and a planning tool.

- **Authentication and Login Flow:**

The system includes login/logout-related components so users can enter and leave the application flow.

- **Responsive User Interface:**

The frontend is designed using component-based React pages and CSS files. The goal is to keep the interface organized, readable, and easy for users to navigate.

1.4 Originality vs Inspiration

The project idea is inspired by common wardrobe, fashion planning, and closet organization apps. However, the implementation is original to our team. Our version focuses on a simple student-friendly interface where the main goal is not advanced fashion recommendation, but practical outfit organization and planning.

Unlike many existing closet apps that may include complex shopping or social media features, our project focuses on the core workflow of storing clothing information, building outfits, saving favorites, and planning outfits by date. This makes the application easier to understand, easier to test, and more appropriate for the course project scope.

2 System Overview and Project Description

2.1 Major Functional Modules

The application is separated into several functional modules:

- **Authentication Module:**

This module handles the login and logout flow. It includes components such as LoginPage and LogoutPage. It allows the application to separate user access from the main outfit and closet pages.

- **Closet and Item Module:**

This module manages clothing item data. It connects frontend item-related views with backend item routes and item models. Users can create, read, update, and delete closet items.

- **Category Module:**

This module manages clothing categories. It allows items to be grouped logically, which improves the organization of the outfit builder.

- **Outfit Module:**

This module handles outfit CRUD operations. Users can create a new outfit, retrieve saved outfits, update outfit details, and delete outfits.

- **Favorites Module:**

This module allows users to save and remove favorite outfits. It improves the user experience by giving users faster access to preferred outfit combinations.

- **Planner Module:**

This module allows users to assign outfits to dates. It connects outfit data with planning information such as date, occasion, and notes.

- **Admin Module:**

This module supports admin-related routes or management pages. It may be used for managing data or testing backend functionality.

- The modular design keeps the project easier to maintain because each feature has its own route files, page files, and styling files. The frontend communicates with the backend through HTTP requests, and the backend communicates with MongoDB through Mongoose models.

2.2 UI and Navigation Flow

A typical user begins on the login or home page. After entering the application, the user can navigate to major pages such as the Home Page, Outfit Builder Page, Outfit Detail Page, Favorites Page, and Outfit Planner Page.

The general navigation flow is:

1. The user opens the application in the browser.
2. The user logs in or reaches the Home Page.
3. From the Home Page, the user can go to outfit-related views.
4. On the Outfit Builder Page, the user creates a new outfit.

5. After saving an outfit, the frontend sends a POST request to the backend.
6. Saved outfits can be displayed through GET requests.
7. The user can open an Outfit Detail Page to view a selected outfit.
8. The user can update outfit information through a PUT request.
9. The user can delete an outfit through a DELETE request.
10. The user can mark outfits as favorites or assign them to planner dates.

Each main page has a specific purpose:

Home Page:

Provides the main entry point and navigation for the application.

Outfit Builder Page:

Allows users to create outfit combinations by selecting or entering clothing items.

Outfit Detail Page:

Displays full information about a selected outfit and supports edit/delete actions.

Favorites Page:

Displays outfits saved by the user as favorites.

Outfit Planner Page:

Allows users to assign outfits to calendar dates and manage planned entries.

FAQ Page:

Provides helpful information about how to use the application.

2.3 CRUD Entities

2.3.1 Outfit Entity

Example fields:

- `_id`: Unique MongoDB identifier
- `name`: Name of the outfit

- items: Clothing items included in the outfit
- notes: Optional outfit notes
- occasion: Optional occasion type
- favorite: Boolean value showing whether the outfit is favorited
- createdAt: Timestamp for creation
- updatedAt: Timestamp for last update

Supported actions:

- **Create:** User creates and saves a new outfit.
- **Read:** User retrieves saved outfits or views one outfit.
- **Update:** User edits outfit name, items, notes, or favorite status.
- **Delete:** User removes an outfit from the system.

These CRUD operations are necessary because outfit creation and management are the main purpose of the application.

2.3.2 Item Entity

Example fields:

- `_id`: Unique MongoDB identifier
- name: Clothing item name
- category: Clothing category
- color: Optional item color
- imageUrl: Optional image path or URL
- description: Optional item details

Supported actions:

- **Create:** Add a new clothing item.
- **Read:** Retrieve clothing items.
- **Update:** Edit item details.
- **Delete:** Remove an item from the closet.

Items are necessary because outfits are built using clothing items.

2.3.3 Category Entity

Example fields:

- `_id`: Unique MongoDB identifier
- `name`: Category name
- `description`: Optional category description

Supported actions:

- **Create:** Add a new category.
- **Read:** Retrieve categories.
- **Update:** Rename or edit a category.
- **Delete:** Remove a category.

Categories help organize closet items and make the outfit builder easier to use.

2.3.4 Planner Entry Entity

Example fields:

- `_id`: Unique MongoDB identifier
- `outfitId`: Reference to an outfit
- `date`: Planned calendar date
- `occasion`: Optional occasion
- `notes`: Optional notes

Supported actions:

- **Create:** Assign an outfit to a date.
- **Read:** Retrieve planner entries.
- **Update:** Change date, occasion, or notes.
- **Delete:** Remove a planner entry.

Planner entries are necessary because the application supports outfit planning, not only outfit storage.

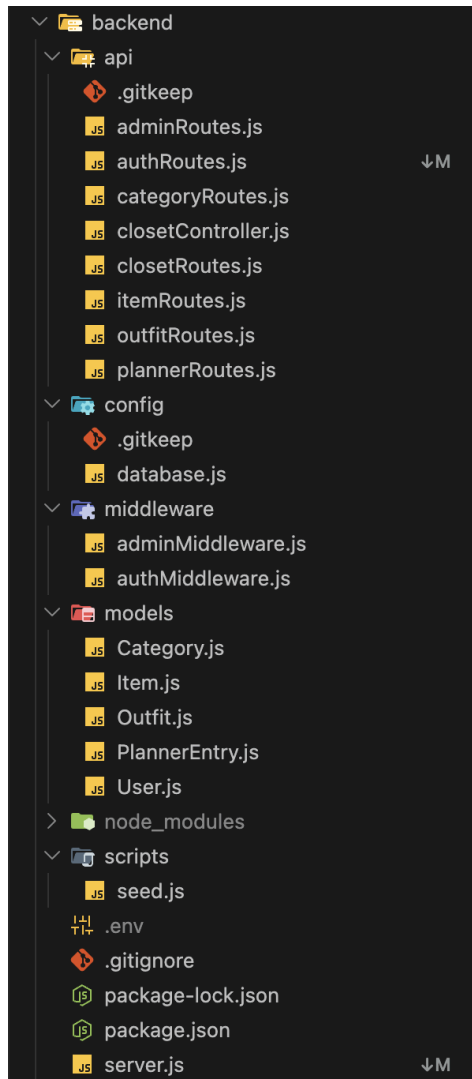
3 File and Folder Architecture

3.1 Folder Structure Overview

- The project is organized into two main directories: frontend/ and backend/. This separation makes the project easier to manage because the user interface and server logic are developed independently.
- The frontend/ folder contains the React application, page components, component files, styling files, and frontend configuration. The backend/ folder contains the Express server, API routes, controllers, database configuration, middleware, and Mongoose models.
- This structure improves maintainability because frontend developers can work on React pages while backend developers can work on API routes and database logic without mixing responsibilities. It also matches common full-stack project organization patterns.

3.2 Backend Folder Structure

3.2.1 Backend Folder Tree



3.2.2 Backend Folder Description

- **server.js**

This is the main backend entry point. It creates the Express application, connects to the database, applies middleware, sets up CORS, registers API routes, and starts the server on the selected port.

- **api/**

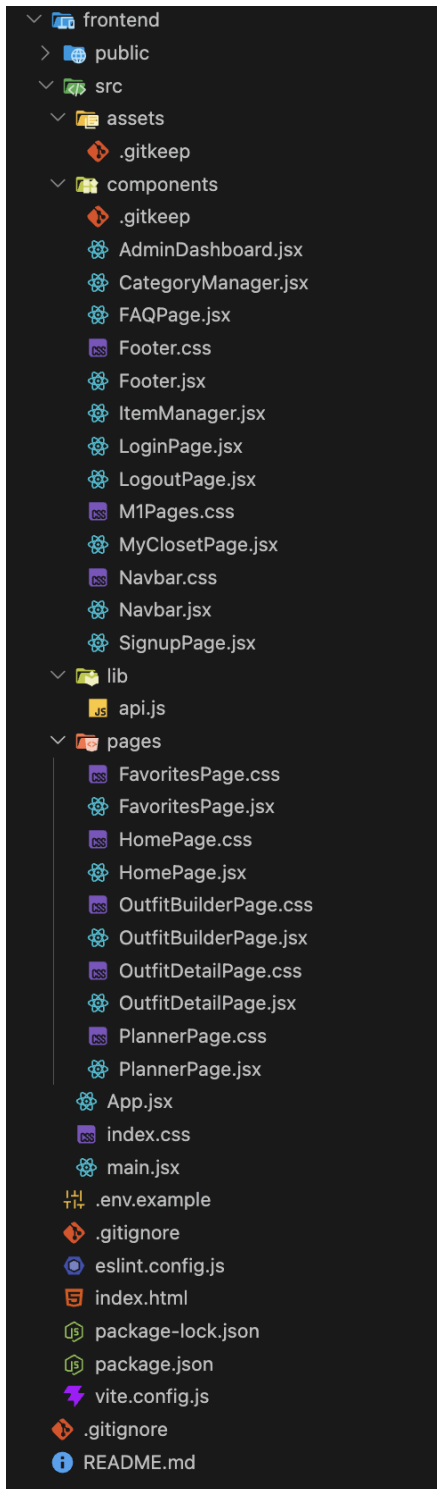
This folder contains backend route files and route-related logic. Each file is separated

by feature, such as authentication, admin, category, closet, item, and outfit routes. This keeps the API organized and easier to debug.

- **adminRoutes.js**
Contains routes related to admin-level actions or admin data management.
- **authRoutes.js**
Contains authentication-related routes such as login or registration depending on the project implementation.
- **categoryRoutes.js**
Contains CRUD endpoints for clothing categories.
- **closetRoutes.js**
Contains endpoints related to the user closet and closet-level data.
- **closetController.js**
Contains controller logic for closet-related operations. Keeping controller logic separate from route declarations improves readability.
- **itemRoutes.js**
Contains CRUD endpoints for clothing items.
- **outfitRoutes.js**
Contains CRUD endpoints for outfits, including create, retrieve, update, and delete operations.
- **config/database.js**
Handles the MongoDB database connection. It keeps connection logic separate from the main server file.
- **middleware/**
Stores middleware functions, such as authentication checks, validation, or error-handling middleware.
- **Models/**
Stores Mongoose schema/model files that define how data is stored in MongoDB.
- **package.json and package-lock.json**
Store backend dependencies, scripts, and package versions.

3.3 Frontend Folder Structure

3.3.1 Frontend Folder Tree



3.3.2 Frontend Folder Description

- **src/**
Contains the main React source code for the frontend application.
- **components/**
Contains reusable or general-purpose React components such as LoginPage, LogoutPage, and FAQPage.
- **pages/**
Contains larger page-level components that represent full views in the application. Examples include HomePage, OutfitBuilder, OutfitDetailPage, and FavoritesPage.
- **CSS files**
Each major page has a CSS file for styling. This keeps styling close to the page it belongs to and makes the project easier to understand.
- **App.jsx**
Controls the main frontend app structure and routing between pages.
- **main.jsx**
Renders the React application into the browser through the root DOM element.
- **index.html**
The HTML entry point used by Vite.
- **vite.config.js**
Stores Vite configuration for frontend development.

4 Code Explanation and Logic Flow

4.1 Frontend–Backend Interaction

The frontend communicates with the backend using HTTP requests. When a user interacts with the React UI, the component calls a backend route using `fetch` or a similar request method. The frontend sends data as JSON in the request body for POST and PUT requests. For GET and DELETE requests, the frontend may use route parameters such as an outfit id.

Example request flow:

1. The user fills out a form on the Outfit Builder Page.
2. React stores the input values in component state using `useState`.
3. When the user submits the form, the component sends a POST request to the backend.
4. The backend route receives the JSON body.
5. The backend validates the required fields.
6. The backend uses a Mongoose model to save the data in MongoDB.
7. The backend returns a success response with the saved object.
8. The frontend updates state and displays the new outfit or a success message.

Error handling is done through try/catch blocks on the frontend and backend. If the backend returns an error response, the frontend can display an error message or fallback UI. Empty states are also useful when there are no saved outfits, no favorites, or no planner entries.

4.2 React Component Example

```
async function handleSave() {
  try {
    if (!outfitName.trim()) {
      setError("Please enter an outfit name.");
      return;
    }

    if (selectedItems.length === 0) {
      setError("Please select at least one item.");
      return;
    }

    setSaving(true);
    setError("");

    const payload = {
      name: outfitName.trim(),
      occasion,
      season,
      notes,
      favorite: true,
      items: selectedItems.map((item) => ({
        id: item.id,
        name: item.name,
        category: item.category,
        color: item.color,
        size: item.size,
        description: item.description || "",
      })),
    };

    const res = await fetch(
      editId ? `${API_URL}/outfits/${editId}` : `${API_URL}/outfits`,
      {
        method: editId ? "PUT" : "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(payload),
      }
    );

    const data = await res.json();

    if (!res.ok) {
      throw new Error(data.error || "Unable to save outfit.");
    }

    setSaved(true);

    setTimeout(() => {
      navigate("/favorites");
    }, 700);
  } catch (err) {
    setError(err.message || "Unable to save outfit.");
  } finally {
    setSaving(false);
  }
}
```

This component demonstrates form handling, state updates, a POST request to the backend, and conditional UI feedback. The `useState` hook stores user input, and `handleSubmit` sends the information to the backend. If the request succeeds, the form clears and a success message appears. If the request fails, an error message appears.

4.3 API Route Examples

Provide one example for each CRUD-related route type (POST, GET, PUT, DELETE). For each route, describe:

- The purpose of the endpoint

Endpoint

POST /api/items

Purpose

This endpoint creates a new closet item and saves it to the MongoDB `item` collection. It is used when a user adds a new clothing item from the Items page.

- Expected request format (body schema, params)

Request body:

```
{
  "name": "Black Hoodie",
  "category": "Tops",
  "color": "Black",
  "size": "M",
  "description": "Comfortable hoodie for cold weather."
}
```

Required fields:

name

category

color

size

- Validation steps performed

Validation Steps Performed

The backend should check that:

name is not empty

Category is not empty

color is not empty

size is not empty

If any required field is missing, the backend returns a 400 BAD REQUEST.

Example validation error:

```
{  
  "error": "Item name, category, color, and size are required."  
}
```

- Any middleware used (auth middleware, validation middleware)

Global middleware from `server.js`:

- `app.use(cors(...));`
- `app.use(express.json());`

- Database operations triggered by the route

The route creates a new document using Mongoose:

```
Item.create({  
  name,  
  category,  
  color,  
  size,
```

```
description
});
```

- Expected response structure and status codes

Expected Response

Success status:

201 Created

Example response:

```
{
  "id": "6a039095ec67948956917f8",
  "name": "Black Hoodie",
  "category": "Tops",
  "color": "Black",
  "size": "M",
  "description": "Comfortable hoodie for cold weather.",
  "createdAt": "2026-05-12T10:30:00.000Z",
  "updatedAt": "2026-05-12T10:30:00.000Z"
}
```

- How errors are handled and returned to the frontend

Error Handling

Errors are caught in a `try/catch` block. If something fails while creating the item, the backend returns:

- 500 Internal Server Error

Example error response:

```
{  
  "error": "Unable to create item."  
}
```

4.4 Database Interaction

Show your schema (Mongoose model or SQL table) and explain:

- The fields stored in the database and their types

1. Item Schema

- **Fields:** name String, category String, color String, size String, description String, imageUrl String.
 - **Validation:** name, category, color, and size are required so incomplete closet items cannot be saved.
 - **Queries:** Uses `find()` to display all closet items, `create()` to add a new item, `findByIdAndUpdate()` to edit an item, and `findByIdAndDelete()` to delete an item.
 - **Optimization:** MongoDB automatically indexes `_id`, which helps find, update, and delete items quickly.
-
- Relationships between collections/tables (if applicable)

The database uses simple relationships between collections.

- **Items and Categories:** Each item stores a `category` field as a string, such as "Tops" or "Shoes". This connects closet items to categories.
- **Outfits and Items:** Each outfit stores an `items` array. This array contains the selected clothing items that make up the outfit.

- **Planner Entries and Outfits:** Each planner entry stores an `outfitId` and `outfitName`, which connects a planned calendar date to a saved outfit.
- **Users and Authentication:** The user collection stores login information. The frontend stores the logged-in user/token so protected pages can be accessed after login.

Example:

```
{  
  
  name: "Coffee Run Look",  
  
  occasion: "Casual",  
  
  season: "All Season",  
  
  items: [  
  
    {  
  
      id: "6a039095ec67948956917f8",  
  
      name: "White T-Shirt",  
  
      category: "Tops"  
  
    }  
  
  ],  
  
  favorite: true  
  
}
```

- Validation rules or constraints at the database level
1. The Mongoose schemas include validation rules to make sure incomplete data is not saved.
 2. **Item Schema:** name, category, color, and size are required.
 3. **Category Schema:** name is required and unique, so duplicate categories are prevented.
 4. **Outfit Schema:** name is required. Other fields like occasion, season, and notes have default empty values.
 5. **Planner Entry Schema:** date, outfitId, and outfitName are required.
 6. **User Schema:** email and passwordHash are required. email is unique, and role is limited to "user" or "admin".
- Common queries used (find, update, aggregate, etc.)

The backend commonly uses Mongoose CRUD queries.

```
Item.find()
```

Used to get all closet items.

```
Item.create(req.body)
```

Used to add a new closet item.

```
Item.findById(req.params.id)
```

Used to get one item or outfit by ID.

```
Item.findByIdAndUpdate(req.params.id, req.body, {  
  new: true,  
  runValidators: true,  
})
```

Used to update an existing item and return the updated version.

```
Item.findByIdAndDelete(req.params.id)
```

Used to delete an item.

```
User.findOne({ email })
```

Used during login/signup to check if an account exists.

```
Outfit.find({ favorite: true })
```

Used to load favorite outfits.

```
PlannerEntry.find({ date: selectedDate })
```

Used to load outfits planned for a selected calendar date.

```
Item.countDocuments()
```

```
Category.countDocuments()
```

```
Outfit.countDocuments()
```

```
User.countDocuments()
```

- Any indexing or query optimization considerations

MongoDB automatically creates an index on `_id` for every document. This makes routes using IDs fast, such as:

```
GET /api/items/:id
```

```
PUT /api/items/:id
```

```
DELETE /api/items/:id
```

Extra useful indexes include:

```
userSchema.index({ email: 1 }, { unique: true });
```

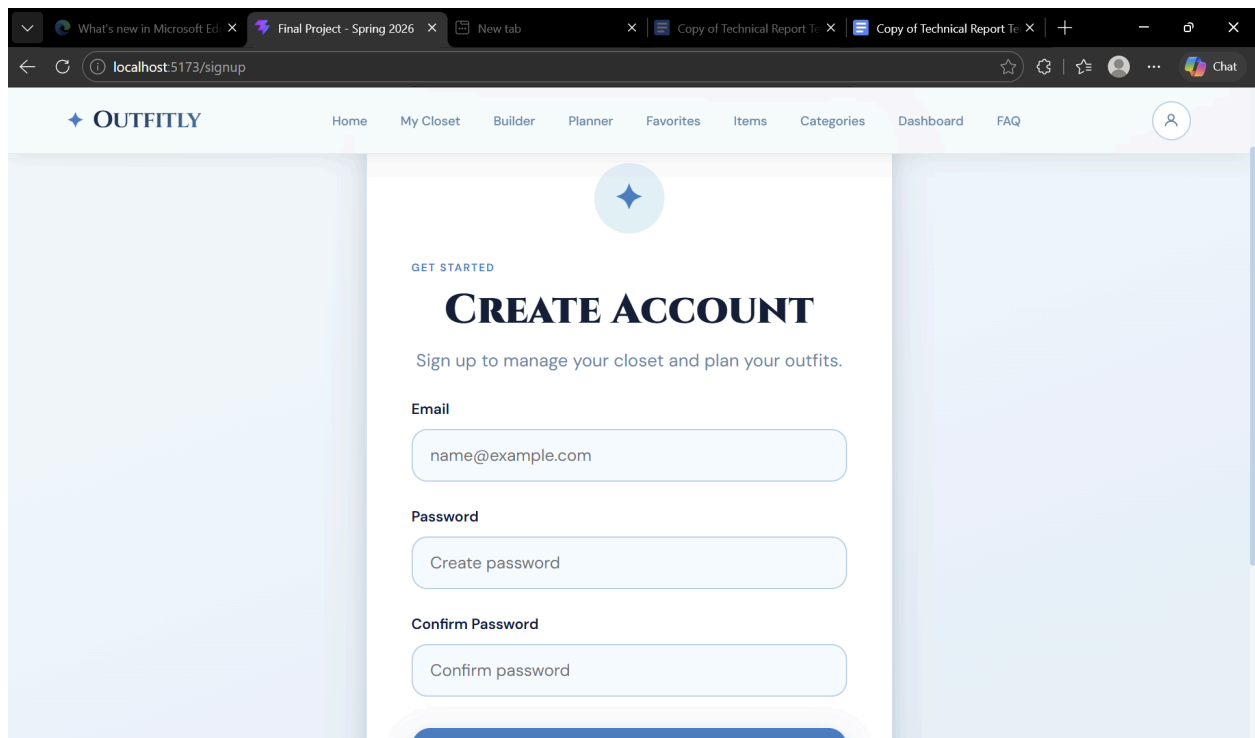
```
categorySchema.index({ name: 1 }, { unique: true });
```

```
outfitSchema.index({ favorite: 1 });  
plannerEntrySchema.index({ date: 1 });
```

5 Screenshots and Web Views

Include screenshots of each important page in the application. For each screenshot:

- Briefly describe the purpose of the page
- Identify key UI elements shown
- Explain what actions the user can take (e.g., submit form, navigate, edit item)
- Describe what happens after interaction (e.g., “Submitting this form sends a POST request to /api/routes and displays a success toast”)
- Note any dynamic behavior such as filtering, searching, or live updates



Purpose:

The Signup page allows a new user to create an account for the Outfitly application.

Key UI elements shown:

The page includes the navigation bar, Outfitly logo, email input, password input, confirm password input, and a Create Account button.

Actions the user can take:

The user can enter an email, create a password, confirm the password, and submit the form. The user can also click the Log in link if they already have an account.

What happens after interaction:

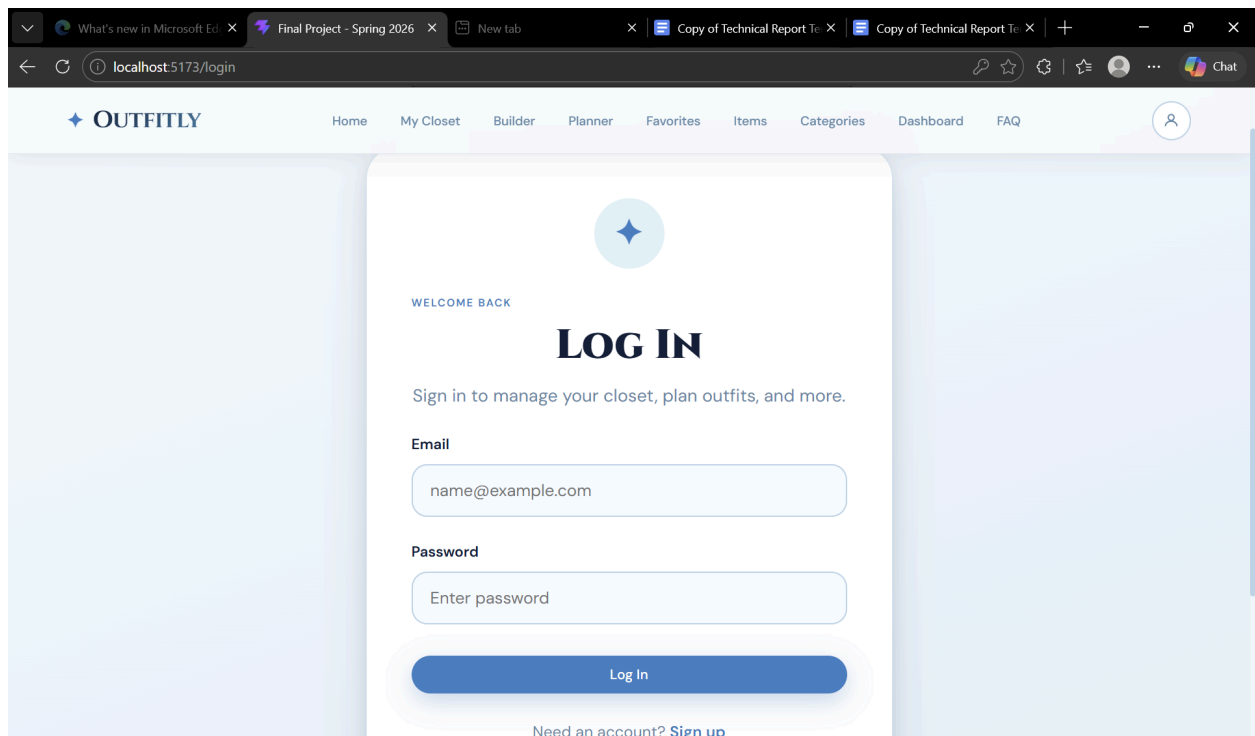
When the user submits the form, the frontend sends a **POST** request to:

POST /api/auth/signup

If signup is successful, the account is created in MongoDB and the user is redirected to the Login page. If the passwords do not match or the email already exists, an error message is displayed.

Dynamic behavior:

The form checks that the password and confirm password match before sending the request. Error messages appear dynamically if validation fails.



The screenshot shows a web browser window with the URL `localhost:5173/login`. The page features a navigation bar with the Outfitly logo and links to Home, My Closet, Builder, Planner, Favorites, Items, Categories, Dashboard, and FAQ. The main content area is a login form with a blue star icon and the text "WELCOME BACK". The form title is "LOG IN" in large, bold, dark blue letters. Below the title, it says "Sign in to manage your closet, plan outfits, and more." The form has two input fields: "Email" with the placeholder "name@example.com" and "Password" with the placeholder "Enter password". A blue "Log In" button is at the bottom of the form. At the very bottom, there is a link that says "Need an account? Sign up".

5.2 Login Page

Purpose:

The Login page allows an existing user to sign in to the application.

Key UI elements shown:

The page includes the email input, password input, Log In button, and Sign up link.

Actions the user can take:

The user can enter their email and password and click Log In. They can also navigate to the Signup page.

What happens after interaction:

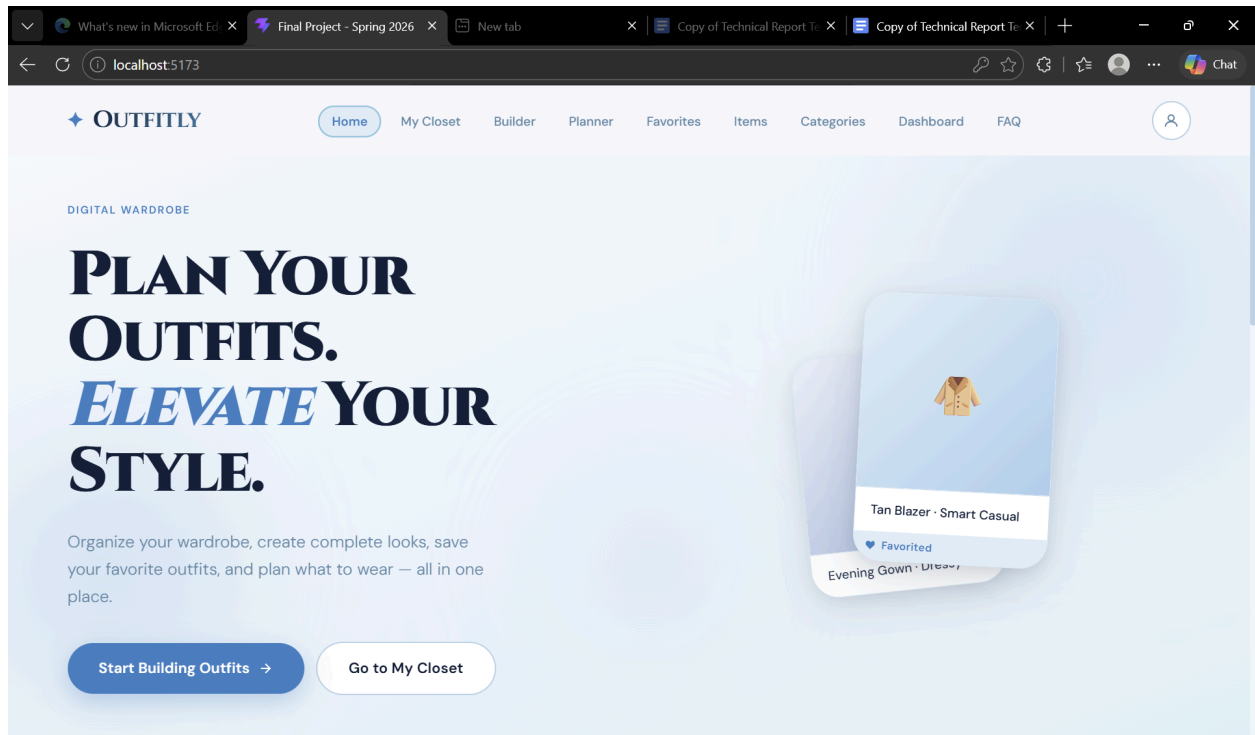
Submitting the form sends a **POST** request to:

`POST /api/auth/login`

If the login is successful, the backend returns user information and a token. The frontend stores the login state and allows the user to access protected pages. If login fails, an error message is shown.

Dynamic behavior:

The page updates based on login status. If the user is already signed in, the app can redirect them away from the login form.



5.4 Home Page

Purpose:

The Home page introduces the Outfitly application and gives users a starting point for navigating the app.

Key UI elements shown:

The page includes the navigation bar, hero heading, description text, Start Building Outfits button, Go to My Closet button, and decorative outfit preview cards.

Actions the user can take:

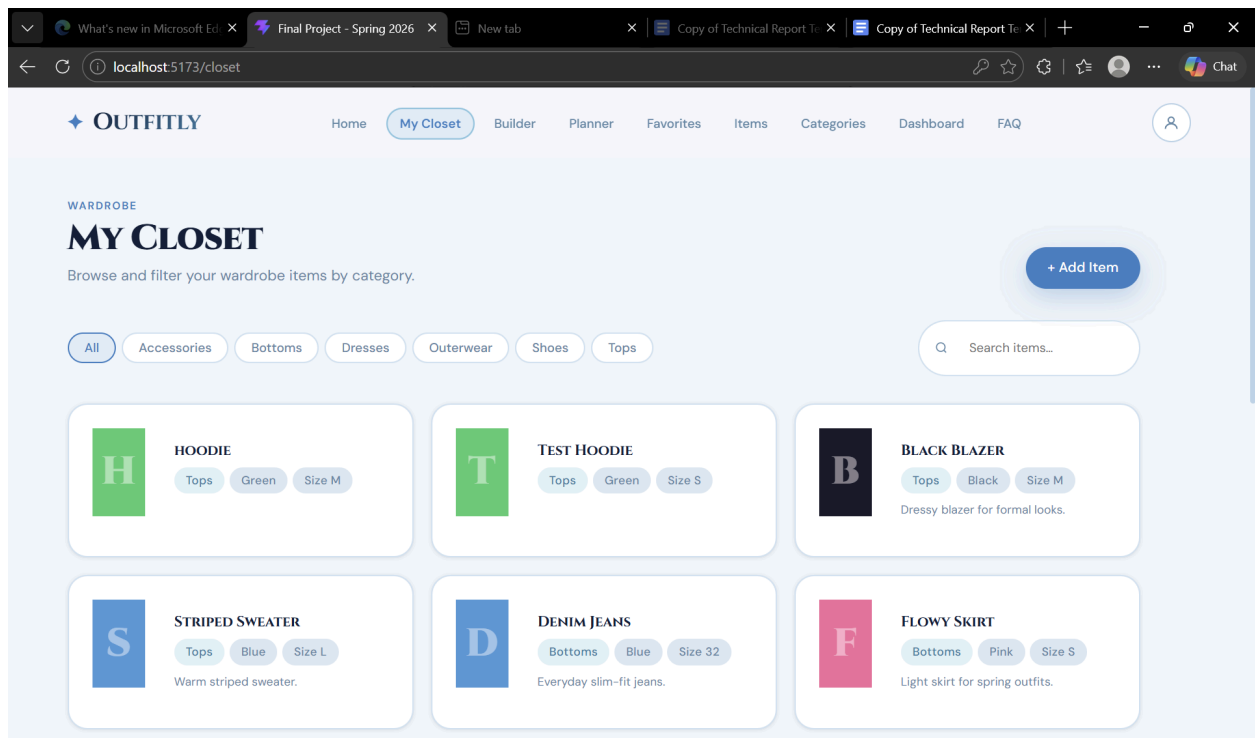
The user can click Start Building Outfits to go to the Builder page or Go to My Closet to open the Closet page.

What happens after interaction:

Navigation is handled by React Router. Clicking a button changes the route without refreshing the full page.

Dynamic behavior:

The active navigation link changes based on the current page.



5.5 My Closet Page

Purpose:

The My Closet page displays all closet items in a visual card layout.

Key UI elements shown:

The page includes category filter buttons, a search bar, item cards, item names, category tags, color tags, and size tags.

Actions the user can take:

The user can search for items, filter items by category, and click Add Item to go to the item management page.

What happens after interaction:

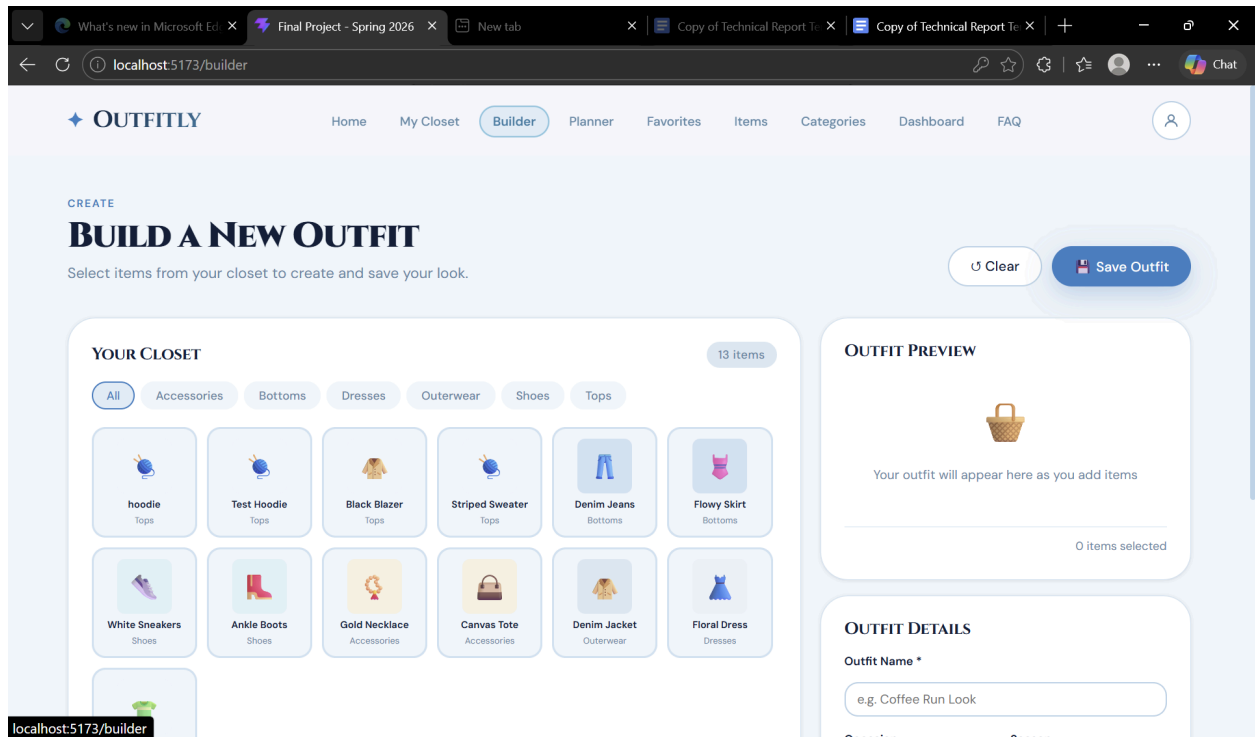
The page loads closet items from the backend using:

GET /api/items

When the user filters or searches, the displayed item cards update on the frontend.

Dynamic behavior:

Filtering and searching update the visible items without reloading the page.



5.6 Builder Page

Purpose:

The Builder page allows users to create a new outfit by selecting clothing items from their closet.

Key UI elements shown:

The page includes category filters, closet item cards, outfit preview section, outfit name input, occasion dropdown, season dropdown, notes field, Clear button, and Save Outfit button.

Actions the user can take:

The user can filter items, select clothing items, remove selected items from the preview, enter outfit details, clear the form, and save the outfit.

What happens after interaction:

When the page loads, it fetches items and categories:

GET /api/items

GET /api/categories

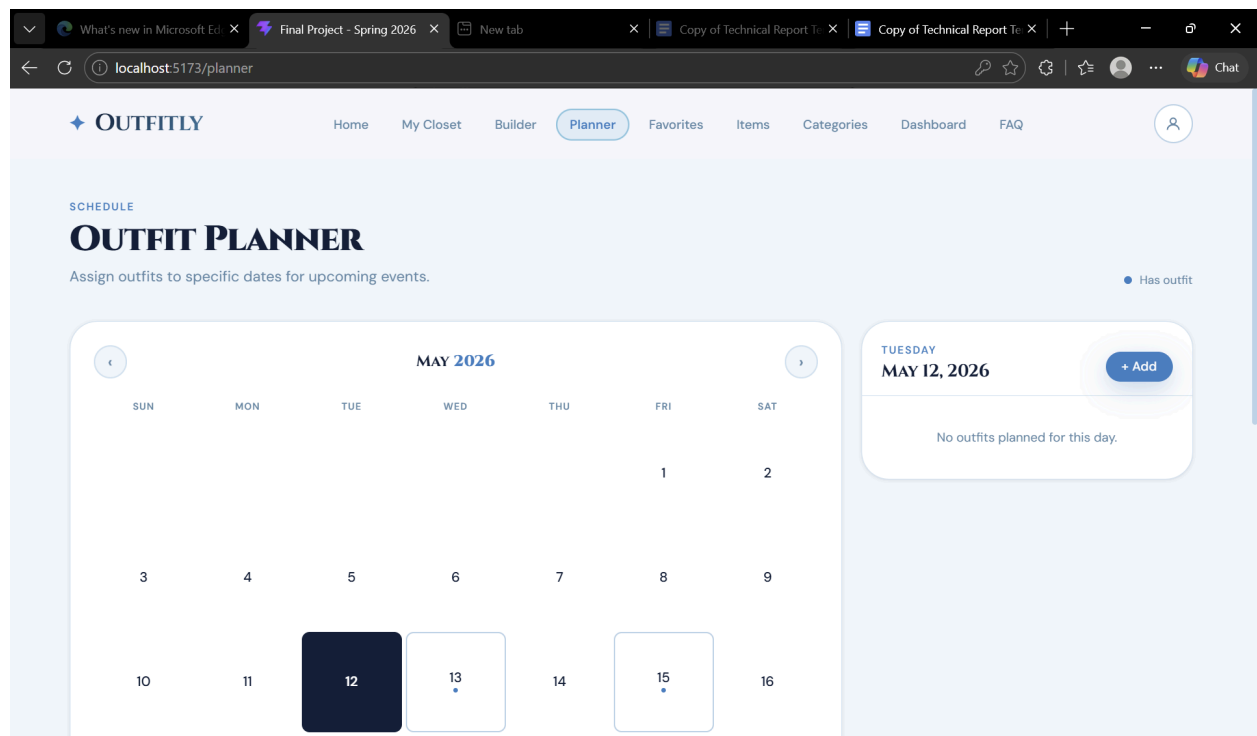
When the user saves an outfit, the frontend sends:

POST /api/outfits

If the outfit is saved successfully, it is stored in MongoDB and can appear on the Favorites page or Planner page depending on the data.

Dynamic behavior:

Selected items immediately appear in the Outfit Preview section. The item count updates live as items are selected or removed.



5.7 Planner Page

Purpose:

The Planner page allows users to assign saved outfits to specific calendar dates.

Key UI elements shown:

The page includes a monthly calendar, selected date panel, Add button, and an indicator showing dates that have planned outfits.

Actions the user can take:

The user can choose a date, add an outfit to that date, move between months, and view planned outfits for selected dates.

What happens after interaction:

The planner loads outfit and planner data from the backend:

GET /api/outfits

GET /api/planner

Adding a planned outfit sends:

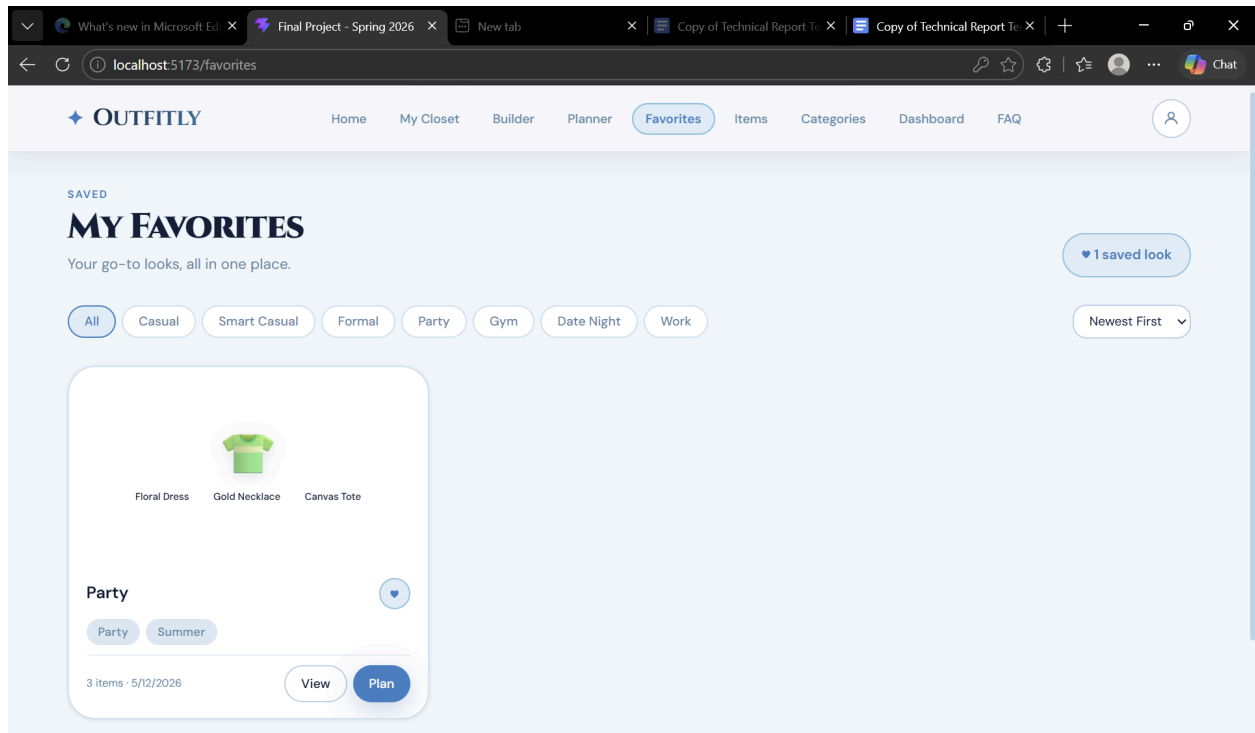
POST /api/planner

Deleting a planned outfit sends:

DELETE /api/planner/:id

Dynamic behavior:

The selected date changes when the user clicks a calendar day. Dates with outfits are visually marked.



5.8 Favorites Page

Purpose:

The Favorites page displays saved or favorited outfits.

Key UI elements shown:

The page includes favorite outfit cards, outfit name, item preview, occasion tag, season tag, View button, Plan button, saved look count, filters, and sorting dropdown.

Actions the user can take:

The user can filter favorite outfits by occasion, sort them, view outfit details, or plan an outfit.

What happens after interaction:

The page loads saved outfits from the backend using:

GET /api/outfits

The frontend filters outfits where **favorite** is **true**. Clicking View opens the outfit detail page. Clicking Plan sends the user to the Planner flow.

Dynamic behavior:

The saved look count updates based on the number of favorite outfits. Filters and sorting change the displayed cards.

The screenshot shows a web browser window with the URL `localhost:5173/items`. The application is 'OUTFITLY'. The navigation bar includes links to Home, My Closet, Builder, Planner, Favorites, Items (highlighted), Categories, Dashboard, and FAQ. The main section is titled 'MANAGE CLOSET ITEM MANAGEMENT' with the subtitle 'Add, edit, view, and delete closet items.' Below this is a form with the following fields: 'Item name', 'Select category' (a dropdown menu), 'Color', 'Size', and 'Description'. At the bottom of the form is an 'Add Item' button. Below the form, there are three item cards. The first card is for 'HOODIE' (Tops · Green · M) with 'Edit' and 'Delete' buttons. The second card is for 'TEST HOODIE' (Tops · Green · S) with 'Edit' and 'Delete' buttons. The third card is for 'BLACK BLAZER' (Tops · Black · M) with the description 'Dressy blazer for formal looks.' and 'Edit' and 'Delete' buttons.

5.9 Items Page

Purpose:

The Items page allows users to manage closet items using CRUD operations.

Key UI elements shown:

The page includes item name input, category dropdown, color input, size input, description field, Add Item button, item list, Edit buttons, and Delete buttons.

Actions the user can take:

The user can add a new item, edit an existing item, delete an item, and view all current closet items.

What happens after interaction:

Adding an item sends:

POST /api/items

Editing an item sends:

PUT /api/items/:id

Deleting an item sends:

DELETE /api/items/:id

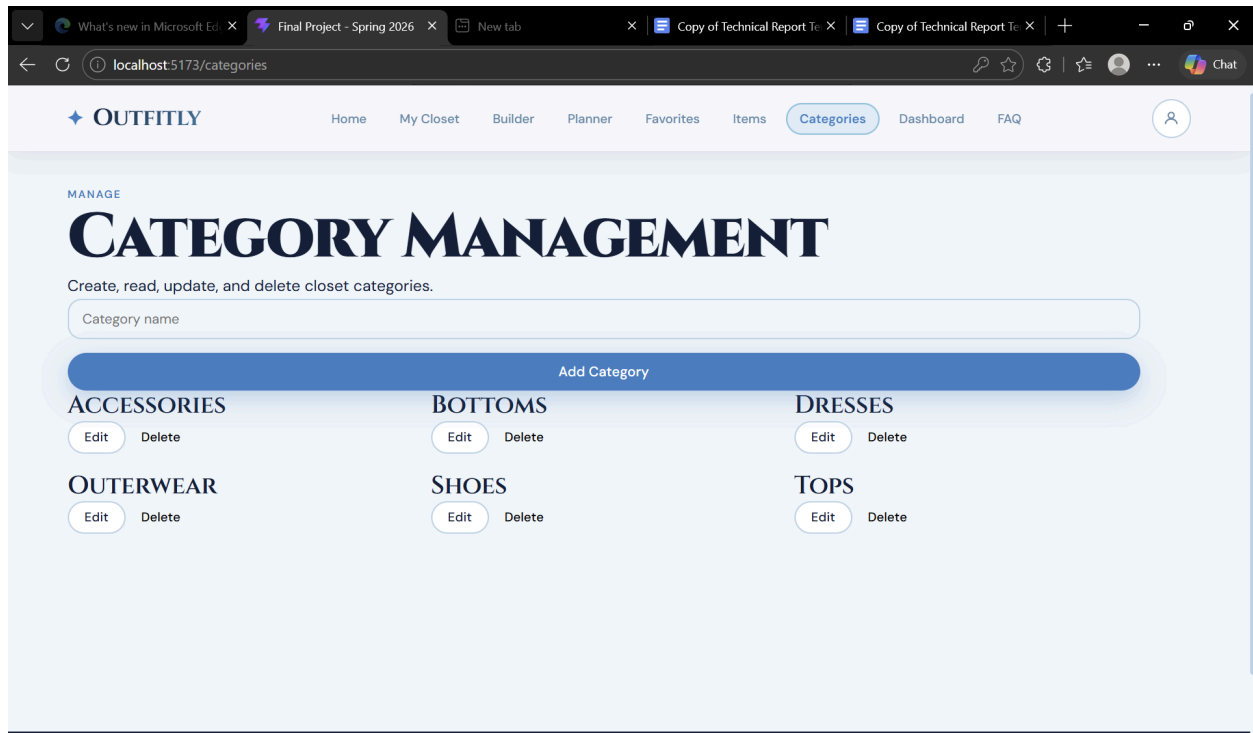
Reading items uses:

GET /api/items

After each successful action, the frontend updates the item list.

Dynamic behavior:

When Edit is clicked, the item data fills the form. After update or delete, the displayed list refreshes.



5.10 Categories Page

Purpose:

The Categories page allows users to manage clothing categories.

Key UI elements shown:

The page includes category name input, Add Category button, category names, Edit buttons, and Delete buttons.

Actions the user can take:

The user can create a new category, edit an existing category, delete a category, and view all categories.

What happens after interaction:

Adding a category sends:

POST /api/categories

Editing a category sends:

PUT /api/categories/:id

Deleting a category sends:

DELETE /api/categories/:id

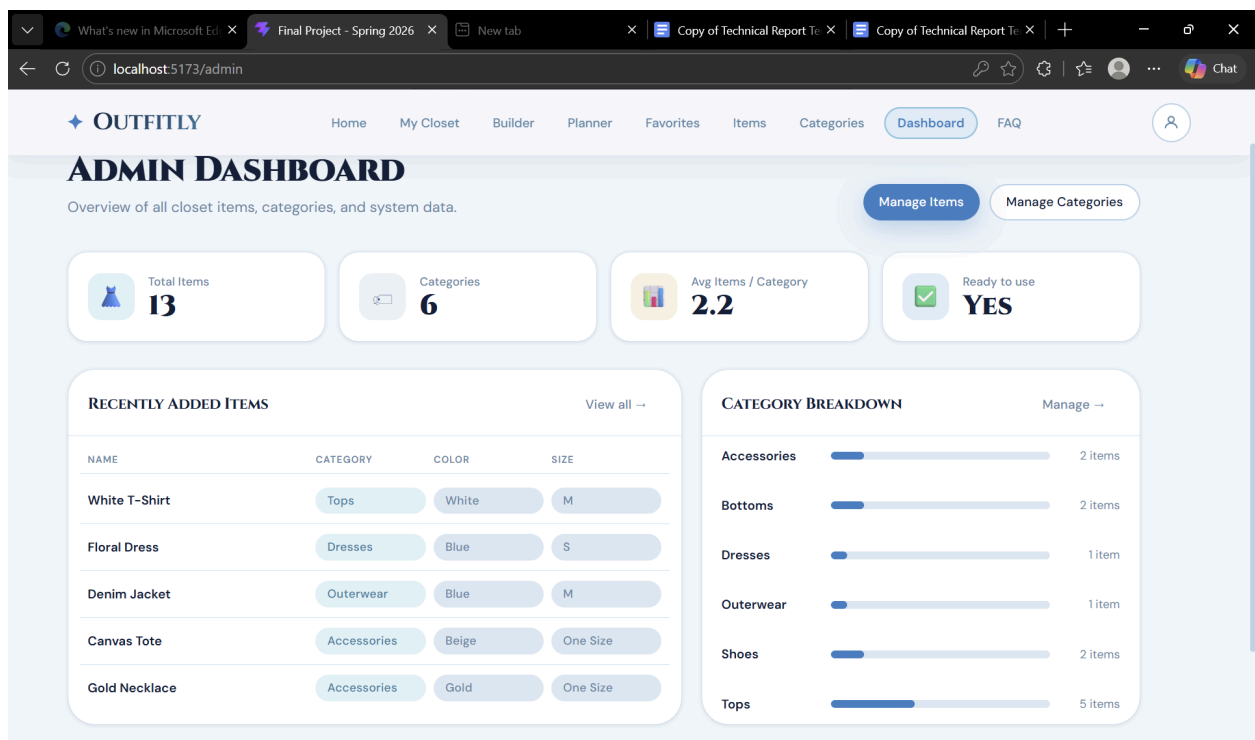
Reading categories uses:

GET /api/categories

After a successful action, the category list updates.

Dynamic behavior:

The category list changes after add, edit, or delete operations. These categories also affect dropdowns and filters on other pages.



5.11 Admin Dashboard Page

Purpose:

The Admin Dashboard gives an overview of the app's data, including total items, categories, and category breakdown.

Key UI elements shown:

The page includes summary stat cards, Manage Items button, Manage Categories button, Recently Added Items table, and Category Breakdown chart/list.

Actions the user can take:

The user can view app statistics and navigate to item or category management pages.

What happens after interaction:

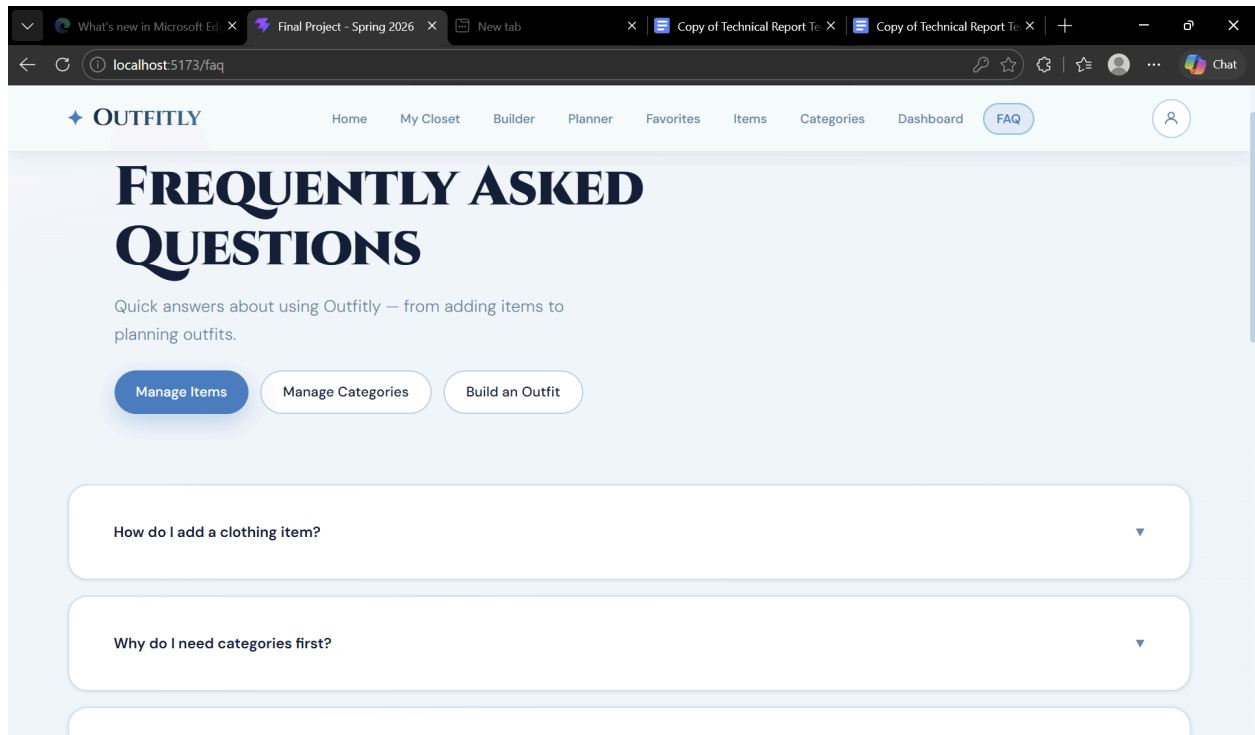
The dashboard loads data from the backend, usually through an admin route such as:

GET /api/admin/stats

The backend performs count queries and returns totals.

Dynamic behavior:

The total item count, category count, average items per category, and category breakdown update based on current database contents.



5.12 FAQ Page

Purpose:

The FAQ page provides quick answers about how to use Outfitly.

Key UI elements shown:

The page includes FAQ question cards, dropdown arrows, and quick navigation buttons such as Manage Items, Manage Categories, and Build an Outfit.

Actions the user can take:

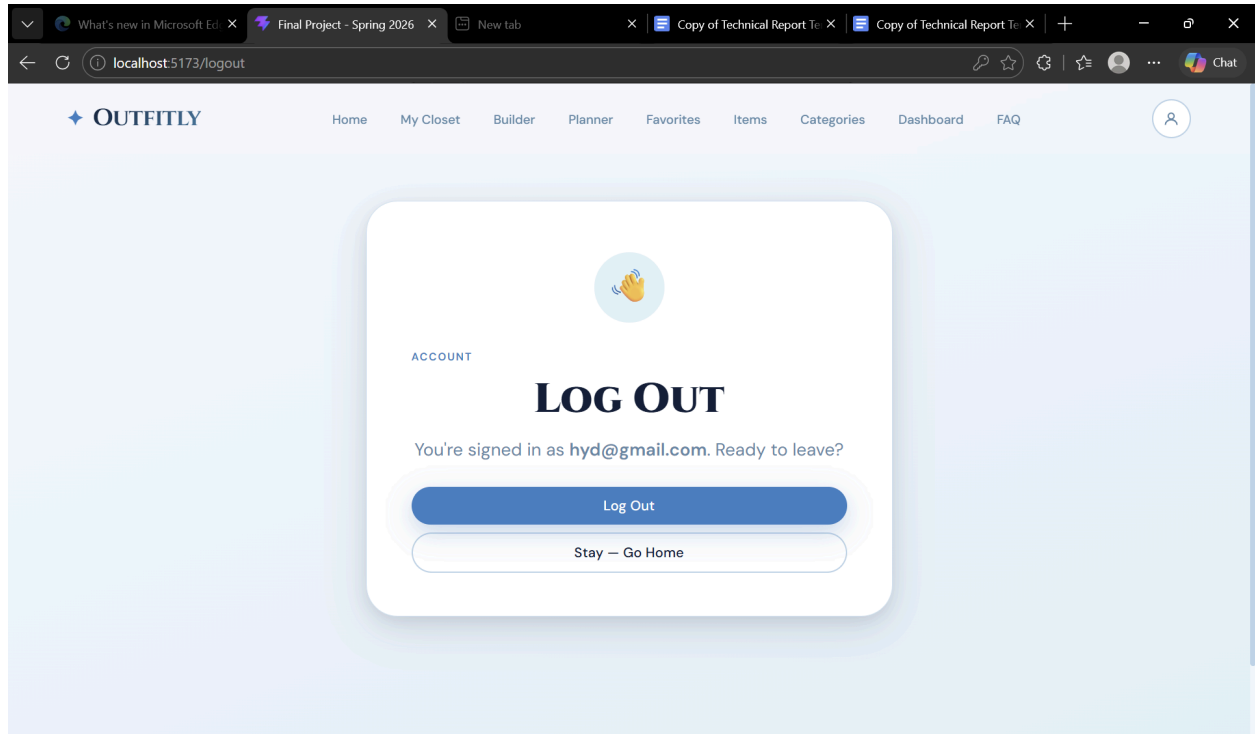
The user can click a question to expand or collapse the answer. They can also use the buttons to navigate to important pages.

What happens after interaction:

FAQ expansion is handled on the frontend with React state. Clicking navigation buttons changes pages using React Router.

Dynamic behavior:

FAQ answers expand and collapse without a backend request.



5.3 Logout Page

Purpose:

The Logout page lets the signed-in user confirm that they want to leave their account session.

Key UI elements shown:

The page shows the logged-in user's email, a Log Out button, and a Stay — Go Home button.

Actions the user can take:

The user can click Log Out to end the session or choose Stay — Go Home to remain signed in and return to the Home page.

What happens after interaction:

Clicking Log Out clears the saved user/token from the frontend, updates the app state, and redirects the user back to the Login page or Home page.

Dynamic behavior:

The page displays the currently signed-in user's email dynamically.

6 Installation and Setup Instructions

6.1 Frontend Setup

Required tools:

- Node.js
- npm
- Vite

Recommended Node version:

Use a stable LTS version of Node.js if possible. Very new Node versions may cause package warnings.

Steps:

cd backend

npm install

npm run dev

The frontend runs on:

http://localhost:5173

If the frontend makes API calls to the backend, the backend base URL should point to:
http://localhost:5000

6.2 Backend Setup

Required tools:

- Node.js
- npm
- nodemon
- dotenv
- MongoDB connection

Steps:

```
cd backend  
npm install  
npm run dev
```

If `npm run dev` is not available, run:
node server.js

The backend server runs on:

http://localhost:5001

6.3 Database Setup

The project uses MongoDB. The database may be run locally through MongoDB Compass or through MongoDB Atlas.

For a local setup:

1. Start MongoDB locally.
2. Create or use the database required by the project.
3. Make sure the backend `.env` file has the correct MongoDB connection string.
4. Run the backend server.
5. Use the application to create data through the frontend, or import seed data if available.

Example database name:

ps16_outfit_planner

Example MongoDB connection string:

mongodb://127.0.0.1:27017/ps16_outfit_planner

6.4 Environment Variables

Example backend .env file:

PORT=5001

MONGO_URI=mongodb://127.0.0.1:27017/ps16_outfit_planner

CLIENT_ORIGIN=http://localhost:5173

Explanation:

- PORT controls the backend server port.
- MONGO_URI stores the MongoDB connection string.
- CLIENT_ORIGIN allows the backend to accept requests from the frontend through CORS.

7 Contribution Overview

7.1 Pulkit Saxena Contributions

- Pulkit worked on the closet, item, category, dashboard, authentication, and backend structure portions of the application. His work focused on organizing the wardrobe data, creating backend routes, connecting MongoDB models, and supporting the pages that provide the data used by the outfit-building workflow.
- Specific responsibilities included:
 - Built the **My Closet** workflow where users can view saved wardrobe or closet data.
 - Implemented the **Items** module so clothing items could be created, read, updated, and deleted.
 - Implemented the **Categories** module so clothing items could be grouped by type such

as tops, bottoms, shoes, accessories, outerwear, and dresses.

- Worked on backend route files such as `itemRoutes.js`, `categoryRoutes.js`, `closetRoutes.js`, and related API files.
- Created or updated MongoDB/Mongoose models for items, categories, closet data, and related records.
- Worked on backend database connection logic through the configuration files.
- Helped connect Express routes to `server.js` so frontend pages could call backend endpoints through `/api/...` routes.
- Worked on dashboard/admin-related functionality for viewing or managing application data.
- Helped with authentication or login-related flow where needed for accessing the application.
- Tested backend API routes and verified that frontend pages could retrieve and display database-backed data.
- Helped debug backend connection issues, route errors, MongoDB setup problems, and integration issues between the frontend and backend.

7.2 Mokshitha Iragamreddy Contributions

Mokshitha worked on the user-facing outfit experience, including the Home Page, Builder Page, Planner Page, Favorites Page, and Outfit Detail workflow. Her work focused on allowing users to create outfits, view outfit information, update or delete outfits, save favorite outfits, and plan outfits for specific dates.

Specific responsibilities included:

- Built the **Home Page** layout and navigation flow for the outfit-related user experience.
- Implemented the **Outfit Builder Page** where users can select closet items and create a new outfit.
- Connected the Builder Page to backend APIs for loading closet items and categories from the database.
- Added category filtering in the Builder Page so users can filter closet items while creating

outfits.

- Added item selection logic so users can select and remove clothing items from the outfit preview.
- Implemented outfit form handling using React state for outfit name, occasion, season, notes, selected items, loading state, saving state, and error messages.
- Added validation so users must enter an outfit name and select at least one item before saving.
- Implemented outfit creation using a **POST** request to the backend outfit API.
- Implemented outfit editing behavior using a **PUT** request when an existing outfit is opened in edit mode.
- Built the **Outfit Detail workflow** where users can view, update, or delete a saved outfit.
- Built the **Favorites Page** so users can view and manage outfits saved as favorites.
- Built the **Planner Page** so users can assign outfits to calendar dates with notes or occasion details.
- Added loading, success, error, disabled-button, and empty-state feedback to improve the user experience.
- Used React routing/navigation so users can move between Home, Builder, Favorites, Planner, and detail views.
- Tested the outfit creation, editing, deleting, favorites, and planner workflows through the browser.
- Helped debug frontend-backend integration issues, including API URL mismatches, CORS behavior, incorrect ports, and merge conflict errors.
- Contributed to the technical report sections related to Member 2 contributions, React component explanation, screenshots, CRUD explanation, and demo preparation.

8 Challenges Faced

8.1 Pulkit Saxena Challenges

- One challenge was organizing the backend routes for multiple parts of the application without making the main server file too crowded. The project had different features for

closet data, items, categories, outfits, dashboard information, and authentication-related behavior. If all of this logic was placed directly inside `server.js`, the backend would have been hard to read and maintain. This was diagnosed while connecting frontend pages to backend endpoints and checking which routes were being called from the browser. The solution was to separate the backend into route files such as `itemRoutes.js`, `categoryRoutes.js`, `closetRoutes.js`, `outfitRoutes.js`, and `adminRoutes.js`, then connect those routes through `server.js`. This made the backend cleaner and easier for both team members to work with.

- Another challenge was making sure the database data matched what the frontend expected. The frontend pages needed item names, categories, colors, sizes, descriptions, and image fields to display closet items correctly. If the backend returned fields with different names or missing values, the UI could show blank sections or fail to load item cards properly. This was diagnosed by checking browser console errors, testing API responses, and comparing the returned JSON data with the frontend component logic. The solution was to keep the item and category data consistent across MongoDB, backend routes, and frontend state. This helped the closet, builder, and category-related pages work together more smoothly.
- A third challenge was connecting dashboard and management pages to real backend data instead of static placeholder content. The dashboard needed to summarize application data from different collections, and the item/category pages needed to show current database records. This required making sure the backend routes returned useful data and that the frontend refreshed correctly after changes. The solution was to test the API routes one by one, verify the MongoDB records, and update frontend calls so that the displayed data came from the backend. This improved the reliability of the application because the UI reflected actual stored data instead of hardcoded examples.

8.2 Mokshitha Iragamreddy Challenges

- One challenge was making the Outfit Builder Page handle both creating a new outfit and editing an existing outfit. These two workflows are similar, but they require

different API behavior. When creating a new outfit, the page needs to send a POST request. When editing an existing outfit, the page needs to load the saved outfit first and then send a PUT request using the outfit id. This was diagnosed by testing the builder page in both normal mode and edit mode and checking whether the correct backend route was being called. The solution was to read the edit id from the URL, load the existing outfit when an edit id exists, and use conditional logic inside `handleSave()` to choose between POST and PUT. This made one component support both create and update behavior.

- Another challenge was keeping the selected closet items and outfit preview synchronized. When users clicked items from the closet, the item had to appear in the preview area. If the user clicked the same item again or removed it from the preview, it needed to disappear from the selected list. This was diagnosed through manual testing by selecting and unselecting different items and checking whether the preview count and item cards updated correctly. The solution was to store selected items in React state and use a `toggleItem()` function that adds an item if it is not selected and removes it if it is already selected. This made the Builder Page more interactive and easier to use.
- A third challenge was handling frontend-backend connection problems. The frontend and backend run on different ports, so if the frontend API URL did not match the backend port, the page could not load items, categories, or outfits. This was diagnosed by looking at terminal errors, browser console messages, and failed network requests. The solution was to make sure the API base URL matched the backend server port and that the backend allowed requests from the frontend through CORS. This helped the Builder, Favorites, and Planner pages communicate correctly with the Express backend.
- Another challenge was dealing with merge conflict markers that were accidentally left inside code files after pulling and merging teammate work. Files such as `server.js`, `LoginPage.jsx`, `LogoutPage.jsx`, and `FAQPage.jsx` had conflict markers like `<<<<<< HEAD, =====, and >>>>>> origin/main`. These caused syntax and parse errors when running the frontend and backend. This was diagnosed

from terminal messages such as “Unexpected token < <” and Vite errors saying “Encountered diff marker.” The solution was to open the affected files, choose the correct code, remove the conflict marker lines, save the files, and rerun the project. This showed how important it is to fully resolve Git conflicts before testing the application.

- A final challenge was improving the user experience across the outfit pages. The Builder Page needed validation, loading messages, error messages, disabled save behavior, and success feedback so users would understand what was happening. Without these states, the page could feel confusing, especially if the user tried to save without entering an outfit name or without selecting any items. This was diagnosed by testing normal and incorrect user actions. The solution was to add validation inside `handleSave()`, store error and saving states with React state, and conditionally update the UI based on whether the page was loading, saving, saved, or showing an error. This made the outfit workflow more stable and user-friendly.

9 Final Reflections

9.1 Pulkit Reflection

- This project improved my confidence with backend development, Express routes, MongoDB, Mongoose models, and organizing a full-stack application. The most rewarding part was seeing the closet, items, categories, and dashboard-related features connect with real database data instead of only using static frontend content. It helped me understand how frontend pages depend on clean backend routes and consistent database fields.
- If starting over, I would plan the backend models and route structure earlier so the frontend and backend could connect more smoothly from the beginning. I would also create clearer API documentation for each route so both team members could easily know what data each endpoint expects and returns. This project strengthened my skills in backend organization, database connection, CRUD route design, API testing, and debugging full-stack integration issues.

9.2 Mokshitha Reflection

- This project improved my confidence with React components, React state management, frontend routing, form handling, API calls, and connecting user-facing pages to backend data. The most rewarding part was building the outfit workflow where a user can select closet items, create an outfit, save it, edit it, view favorites, and plan outfits for specific dates. It was satisfying to see user actions on the frontend create real changes through the backend and database.
- If starting over, I would plan the full outfit data flow earlier, especially how outfits, favorites, planner entries, and closet items connect to each other. I would also set up a cleaner Git workflow with my teammate before merging code so we could avoid merge conflicts and broken files. This project helped me gain practical experience with React hooks, controlled forms, CRUD operations, frontend-backend integration, validation, error handling, and improving the user experience of a full-stack web application.